Technical Guide: QR Decomposition via Gram-Schmidt with Column Operations

Linear Solve Framework

June 6, 2026

Abstract

This technical guide presents a comprehensive implementation of QR decomposition using the Gram-Schmidt process expressed through elementary column operation matrices. The framework transforms a matrix $A \in \mathbb{R}^{m \times n}$ into an orthogonal matrix Q and an upper triangular matrix R such that $A = QR$. Each elementary operation is represented as right-multiplication by an elementary matrix, providing a mathematically rigorous foundation for the implementation.

Contents

1 Introduction

1.1 Problem Statement

Given a matrix $A \in \mathbb{R}^{m \times n}$ with $m \geq n$, the QR decomposition factors A into:

$$A = Q \times R$$

where:

- $Q \in \mathbb{R}^{m \times n}$ has orthonormal columns: $Q^T Q = I_n$
- $R \in \mathbb{R}^{n \times n}$ is upper triangular

1.2 Applications

- Solving linear least squares problems: $\min \|Ax - b\|_2$
- Computing eigenvalues via the QR algorithm
- Solving linear systems with improved numerical stability
- Computing matrix determinants: $\det(A) = \prod_{i=1}^{n} R_{ii}$

1.3 Approach Overview

The implementation uses the Classical Gram-Schmidt process where each column is successively orthogonalized against all previous columns and normalized. Each operation is expressed as right-multiplication by an elementary matrix, allowing systematic accumulation of the R matrix.

2 Mathematical Foundations

2.1 Elementary Matrices

2.1.1 Elementary Matrix $E_{i,j}$

The matrix $E_{i,j} \in \mathbb{R}^{n \times n}$ has a single 1 at position (i, j) and zeros elsewhere:

$$(E_{i,j})_{kl} = \delta_{ik}\delta_{jl} = \begin{cases} 1 & \text{if } k = i \text{ and } l = j \\ 0 & \text{otherwise} \end{cases}$$

2.1.2 Column Scaling Operation

The operation $C_i \leftarrow \lambda C_i$ is represented by:

$$\text{Scale}_i(\lambda) = I - E_{ii} + \lambda E_{ii} = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & \lambda & & \\ & & & \ddots & \\ & & & & 1 \end{pmatrix}$$

2.1.3 Column Addition Operation

The operation $C_i \leftarrow C_i + \lambda C_j$ is represented by:

$$\text{Add}_{i,j}(\lambda) = I + \lambda E_{ji} = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & \cdots & \lambda \\ & & & \ddots & \\ & & & & 1 \end{pmatrix}$$

2.1.4 Column Swap Operation

The operation $C_i \leftrightarrow C_j$ is represented by:

$$\text{Swap}_{i,j} = I - E_{ii} - E_{jj} + E_{ij} + E_{ji}$$

2.2 Properties of Elementary Matrices

[Invertibility] All elementary operation matrices are invertible:

$$\begin{aligned} \text{Scale}_i(\lambda)^{-1} &= \text{Scale}_i(1/\lambda), \quad \lambda \neq 0 \\ \text{Add}_{i,j}(\lambda)^{-1} &= \text{Add}_{i,j}(-\lambda) \\ \text{Swap}_{i,j}^{-1} &= \text{Swap}_{i,j} \end{aligned}$$

Proof. Direct verification through matrix multiplication. □

2.3 Gram-Schmidt Process

Given a matrix $A = [a_1, a_2, \ldots, a_n]$ with columns $a_j \in \mathbb{R}^m$, the Classical Gram-Schmidt algorithm computes:

2.3.1 Orthogonalization Step

For $j = 1, \ldots, n$:

$$v_j = a_j - \sum_{k=1}^{j-1} \text{proj}_{q_k}(a_j)$$

where the projection is:

$$\text{proj}_{q_k}(a_j) = (q_k^T a_j) q_k$$

2.3.2 Normalization Step

$$q_j = \frac{v_j}{\|v_j\|_2}$$

2.4 Matrix Formulation

Let R be the upper triangular matrix with entries:

$$R_{kj} = \begin{cases} q_k^T a_j & \text{for } k < j \\ \|v_j\|_2 & \text{for } k = j \\ 0 & \text{for } k > j \end{cases}$$

Then:

$$A = QR$$

3 Algorithm Description

3.1 Classical Gram-Schmidt Algorithm

Algorithm 1 Classical Gram-Schmidt

```
procedure GramSchmidt(A)
    m, n ← rows(A), cols(A)
    Q ← copy(A)
    R ← I_n
    for j ← 0 to n − 1 do
        for k ← 0 to j − 1 do
            R_kj ← Q[:, k]^T · Q[:, j]
            Q[:, j] ← Q[:, j] − R_kj · Q[:, k]
        end for
        R_jj ← ‖Q[:, j]‖_2
        Q[:, j] ← Q[:, j]/R_jj
    end for
    return (Q, R)
end procedure
```

3.2 Column Operation Formulation

The same algorithm expressed with elementary matrices:

3.3 Proof of Correctness

[QR Decomposition Validity] The algorithm produces matrices Q and R satisfying:

1. $Q^T Q = I_n$ (orthonormal columns)
2. R is upper triangular
3. $A = QR$

Algorithm 2 Gram-Schmidt with Column Operations

procedure GramSchmidtColumnOps(A)
  $Q \leftarrow A$
  $R \leftarrow I_n$
  for $j \leftarrow 0$ to $n-1$ **do**
    for $k \leftarrow 0$ to $j-1$ **do**
      $dot \leftarrow Q[:,k]^T \cdot Q[:,j]$
      $Q \leftarrow Q \times \text{Add}_{j,k}(-dot)$
      $R_{kj} \leftarrow dot$
    end for
    $norm \leftarrow \|Q[:,j]\|_2$
    $Q \leftarrow Q \times \text{Scale}_j(1/norm)$
    $R_{jj} \leftarrow norm$
  end for
  return (Q, R)
end procedure

Proof. Let $E_1, E_2, \ldots, E_t$ be the elementary matrices applied in order. Then:

$$Q = A \times E_1 \times E_2 \times \cdots \times E_t$$

Let $R^{-1} = E_1 \times E_2 \times \cdots \times E_t$. Then:

$$Q = A \times R^{-1} \implies A = Q \times R$$

The upper triangular structure of R follows from the order of operations: each operation only modifies column j using previous columns $k < j$. □

4 Implementation Architecture

4.1 Class Hierarchy

```
Matrix (abstract)
 SquareMatrix (abstract)
     Identity
     Eij
     Elementary Operations
         Ci_lamda_Ci
         Ci_plus_lamda_Cj
```

4.2 Core Classes

4.2.1 Matrix Class

Base class for all matrix operations.

```
public class Matrix {
    protected int m;           // number of rows
    protected int n;           // number of columns
```

```
    protected Double[][] t;    // data storage

    public Matrix(int m, int n);
    public Double get(int i, int j);
    public void set(int i, int j, Double value);
}
```

4.2.2 Eij Class

Represents the elementary matrix $E_{i,j}$.

```
public class Eij extends SquareMatrix {
    public Eij(int i, int j, int n, int m) {
        super(n);
        this.t[i][j] = 1.0;
    }
}
```

4.2.3 Ci_plus_lamda_Cj Class

Implements $I + \lambda E_{j,i}$ for column addition.

```
public class Ci_plus_lamda_Cj extends SquareMatrix {
    public Ci_plus_lamda_Cj(int n, Double lambda, int i, int j) {
        super(n);
        Eij eji = new Eij(j, i, n, n);
        Identity id = new Identity(n);
        this.setT(add(multiplyByScalar(eji, lambda), id).getT());
    }
}
```

4.2.4 Ci_lamda_Ci Class

Implements $I - E_{ii} + \lambda E_{ii}$ for column scaling.

```
public class Ci_lamda_Ci extends SquareMatrix {
    public Ci_lamda_Ci(int n, Double lambda, int i) {
        super(n);
        Eij eii = new Eij(i, i, n, n);
        Identity id = new Identity(n);
        Matrix scaled = multiplyByScalar(eii, lambda);
        Matrix result = add(add(id, multiplyByScalar(eii, -1.0)), scaled);
        this.setT(result.getT());
    }
}
```

4.3 QR Decomposition Implementation

```
public class ApplyingGramSchmidtProcess {
    private Matrix Q;           // Orthogonal matrix
    private Matrix R;           // Upper triangular matrix
    private Matrix workingMatrix;

    public void apply() {
        workingMatrix = copy(initialMatrix);
        R = new Identity(n);

        for (int j = 0; j < n; j++) {
            // Orthogonalization phase
            for (int k = 0; k < j; k++) {
                double dot = computeDotProduct(workingMatrix, j, k);

                Ci_plus_lamda_Cj op = new Ci_plus_lamda_Cj(n, -dot, j, k);
                workingMatrix = multiply(workingMatrix, op);

                R.set(k, j, dot);
            }

            // Normalization phase
            double norm = computeNorm(workingMatrix, j);

            Ci_lamda_Ci op = new Ci_lamda_Ci(n, 1.0/norm, j);
            workingMatrix = multiply(workingMatrix, op);

            R.set(j, j, norm);
        }

        Q = workingMatrix;
    }
}
```

5 Complexity Analysis

5.1 Time Complexity

5.2 Space Complexity

- Storage for Q: $O(mn)$
- Storage for R: $O(n^2)$
- Storage for working matrix: $O(mn)$
- Elementary matrices: $O(n^2)$ each (created and discarded)
- **Total space complexity**: $O(mn + n^2)$

Operation	Complexity
Dot product computation	$O(m)$
Column update	$O(m)$
Normalization	$O(m)$
Total per inner loop	$O(m)$
Number of inner loops	$\sum_{j=1}^{n}(j-1) = \frac{n(n-1)}{2}$
Number of outer loops	n
Total complexity	$O(mn^2)$

Table 1: Time Complexity Analysis

6 Numerical Considerations

6.1 Stability Analysis

Classical Gram-Schmidt (CGS) can suffer from loss of orthogonality due to floating-point roundoff errors. The loss of orthogonality satisfies:

$$\|I - Q^T Q\|_2 \leq \kappa_2(A) \cdot \varepsilon$$

where $\kappa_2(A) = \|A\|_2 \|A^{-1}\|_2$ is the condition number and ε is machine epsilon.

6.2 Modified Gram-Schmidt

For better numerical stability, Modified Gram-Schmidt (MGS) is recommended:

Algorithm 3 Modified Gram-Schmidt

```
procedure ModifiedGramSchmidt(A)
    Q ← A
    R ← 0
    for j ← 0 to n − 1 do
        R_jj ← ‖Q[:, j]‖_2
        Q[:, j] ← Q[:, j]/R_jj
        for k ← j + 1 to n − 1 do
            R_jk ← Q[:, j]^T · Q[:, k]
            Q[:, k] ← Q[:, k] − R_jk · Q[:, j]
        end for
    end for
    return (Q, R)
end procedure
```

6.3 Tolerance and Zero Detection

```
public static final double EPSILON = 1e-12;

private boolean isZero(double value) {
    return Math.abs(value) < EPSILON;
}
```

7 Verification and Testing

7.1 Orthogonality Test

$$\text{Verify: } \|Q^T Q - I_n\|_F < \varepsilon$$

```
public boolean isOrthogonal(Matrix Q, double tolerance) {
    Matrix QtQ = multiply(transpose(Q), Q);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            double expected = (i == j) ? 1.0 : 0.0;
            if (Math.abs(QtQ.get(i, j) - expected) > tolerance) {
                return false;
            }
        }
    }
    return true;
}
```

7.2 Reconstruction Test

$$\text{Verify: } \|A - QR\|_F < \varepsilon$$

```
public double reconstructionError(Matrix A, Matrix Q, Matrix R) {
    Matrix QR = multiply(Q, R);
    return frobeniusNorm(subtract(A, QR));
}
```

7.3 Upper Triangular Test

```
public boolean isUpperTriangular(Matrix R, double tolerance) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (Math.abs(R.get(i, j)) > tolerance) {
                return false;
            }
        }
    }
    return true;
}
```

8 Usage Examples

8.1 Basic Usage

```
// Initialize with dimensions
int m = 5, n = 5;
ApplyingGramSchmidtProcess qr = new ApplyingGramSchmidtProcess(10, m, n);
```

```
// Compute decomposition
qr.apply();

// Access results
Matrix Q = qr.getQ();
Matrix R = qr.getR();

// Verify decomposition
Matrix A = qr.getInitialMatrix();
Matrix reconstructed = multiply(Q, R);

// Solve linear system using QR
// Ax = b  =>  QRx = b  =>  Rx = Q^T b
Matrix b = new Matrix(m, 1);
Matrix Qt = transpose(Q);
Matrix Qt_b = multiply(Qt, b);
Matrix x = solveUpperTriangular(R, Qt_b);
```

8.2 Least Squares Solution

```
public Matrix leastSquares(Matrix A, Matrix b) {
    // Perform QR decomposition
    GramSchmidt qr = new GramSchmidt();
    qr.decompose(A);

    Matrix Q = qr.getQ();
    Matrix R = qr.getR();

    // Compute Q^T b
    Matrix Qt_b = multiply(transpose(Q), b);

    // Solve R x = Q^T b by back substitution
    return backSubstitute(R, Qt_b);
}
```

9 Performance Optimization

9.1 In-Place Computation

To reduce memory allocation:

```
public void decomposeInPlace(Matrix A) {
    int n = A.getN();
    this.R = new Identity(n);

    for (int j = 0; j < n; j++) {
        for (int k = 0; k < j; k++) {
```

```
            double dot = dotProduct(A, j, k);

            // Update column in place
            for (int i = 0; i < m; i++) {
                A.set(i, j, A.get(i, j) - dot * A.get(i, k));
            }

            R.set(k, j, dot);
        }

        double norm = norm(A, j);

        // Scale column in place
        for (int i = 0; i < m; i++) {
            A.set(i, j, A.get(i, j) / norm);
        }

        R.set(j, j, norm);
    }

    this.Q = A;
}
```

9.2 Blocked Algorithm

For large matrices, consider the blocked Gram-Schmidt algorithm:

$$A = [A_1, A_2, \ldots, A_b]$$

Process blocks of columns to improve cache locality.

10 Limitations and Alternatives

10.1 Limitations

- Classical Gram-Schmidt loses orthogonality for ill-conditioned matrices
- Requires $m \geq n$ (full column rank)
- Not suitable for rank-deficient matrices without modification

10.2 Alternative Methods

11 Conclusion

This technical guide presents a complete implementation of QR decomposition using the Gram-Schmidt process expressed through elementary column operation matrices. The framework provides:

Method	Stability	Complexity
Classical Gram-Schmidt	Poor	$O(mn^2)$
Modified Gram-Schmidt	Good	$O(mn^2)$
Householder Reflectors	Excellent	$O(mn^2)$
Givens Rotations	Excellent	$O(mn^2)$

Table 2: Comparison of QR Decomposition Methods

- Mathematical rigor through elementary matrix formulations
- Modular design with clear separation of concerns
- Comprehensive verification procedures
- Practical usage examples for linear systems and least squares

The implementation successfully decomposes any full-rank matrix $A \in \mathbb{R}^{m \times n}$ into Q (orthonormal columns) and R (upper triangular) such that $A = QR$, with verification confirming orthogonality, triangular structure, and reconstruction accuracy.

A Complete Code Listing

A.1 Matrix Multiplication

```
public static Matrix multiplyMatrix(Matrix A, Matrix B) {
    int m = A.getM(), n = A.getN(), p = B.getN();
    Matrix C = new Matrix(m, p);

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < p; j++) {
            double sum = 0.0;
            for (int k = 0; k < n; k++) {
                sum += A.get(i, k) * B.get(k, j);
            }
            C.set(i, j, sum);
        }
    }
    return C;
}
```

A.2 Matrix Transpose

```
public static Matrix transposing(Matrix m) {
    Matrix transpose = new Matrix(m.getN(), m.getM());
    for (int i = 0; i < m.getN(); i++) {
        for (int j = 0; j < m.getM(); j++) {
            transpose.set(i, j, m.get(j, i));
        }
    }
```

```
    return transpose;
}
```

B Glossary

- **QR Decomposition**: Factorization of a matrix into orthogonal and upper triangular parts
- **Gram-Schmidt**: Algorithm for orthogonalizing a set of vectors
- **Elementary Matrix**: Matrix representing a single row/column operation
- **Orthogonal Matrix**: Matrix satisfying $Q^TQ = QQ^T = I$
- **Upper Triangular**: Matrix with zeros below the main diagonal
- **Condition Number**: Measure of sensitivity to input perturbations

C References

1. Golub, G. H., & Van Loan, C. F. (2013). Matrix Computations (4th ed.). Johns Hopkins University Press.
2. Trefethen, L. N., & Bau III, D. (1997). Numerical Linear Algebra. SIAM.
3. Björck, Å. (1996). Numerical Methods for Least Squares Problems. SIAM.